

1. Method Hiding vs. Overriding

Static methods **cannot** be overridden. If a subclass defines a static method with the exact same signature as one in the superclass, it is called **Method Hiding**.

- **The behavior:** The method called is determined by the **reference type** (decided at compile time), not the actual object type (runtime).
- **Example:**

java

```
Parent p = new Child();  
p.staticMethod(); // Calls Parent's version, even though the object is  
a Child
```

Use code with caution.

2. Covariant Return Types

From Java 5 onwards, an overriding method can return a **subtype** of the return type declared in the parent method.

- **Rule:** If `Parent.get()` returns `Object`, `Child.get()` can return `String`.
- **Constraint:** You cannot return a supertype (e.g., if parent returns `ArrayList`, child cannot return `List`).

3. Access Modifier Visibility

An overriding method **cannot reduce the visibility** of the parent method, but it can expand it.

- **Hierarchy:** `private < default < protected < public`.
- **Allowed:** Overriding a `protected` method with a `public` one.
- **Forbidden:** Overriding a `public` method with a `protected` or `private` one (Compile-time error).

4. Exception Handling Rules

Subclasses are restricted in what **checked exceptions** they can throw:

- **Checked Exceptions:** The overriding method can only throw the **same exception** or a **narrower (subclass) exception**. It cannot throw broader exceptions (e.g., if parent throws `IOException`, child cannot throw `Exception`).

- **Unchecked Exceptions:** There are **no restrictions** on `RuntimeException` or its subclasses. The overriding method can throw any unchecked exception even if the parent throws none.

5. Private and Final Methods

- **Private Methods:** These are not inherited and thus **cannot be overridden**. If you define a method with the same signature in a child class, it is just a completely separate method with no polymorphic link.
- **Final Methods:** These are inherited but **cannot be modified** or overridden in any subclass.

Summary of Differences

Feature	Overriding	Hiding (Static)
Binding Time	Runtime (Dynamic)	Compile-time (Static)
Determination	Based on Object type	Based on Reference type
Annotation	<code>@Override</code> is valid	<code>@Override</code> causes an error